Sustentabilidade API

Pipeline de CI/CD, Containerização e Orquestração com Docker Compose e GitHub Actions

Tema: Cidades ESG Inteligentes - Eixo Ambiental

Integrantes:

Marcos Alexandre Gomes de Souza - RM 562294

Guilherme Romera - RM 562848

Rafael Camelo Gavarron - RM 561945

FIAP - Disciplina de DevOps / Cloud Computing

2026

1. Visão geral do projeto

A Sustentabilidade API é uma aplicação Java/Spring Boot desenvolvida para o eixo ambiental do tema ESG (Environmental, Social and Governance). Ela permite que empresas se cadastrem, se autentiquem via JWT, registrem emissões de carbono e gerenciem agendamentos de ações de redução desses impactos, com isolamento total de dados entre empresas distintas.

O objetivo desta atividade foi evoluir o projeto para incluir um pipeline de CI/CD funcional, containerizar a aplicação e orquestrar sua infraestrutura (aplicação + banco de dados), entregando evidências reais de build, testes e deploy em dois ambientes (staging e produção).

1.1 Stack tecnológica

- Linguagem/Framework: Java 21 + Spring Boot 3.5 (Web, Data JPA, Security, Validation, Actuator)
- Banco de dados: Oracle Database (via imagem gratuita gvenzl/oracle-free em Docker)
- Migrations: Flyway
- Autenticação: JWT (com.auth0:java-jwt)
- Build: Maven
- Containerização: Docker (multi-stage build)
- Orquestração: Docker Compose
- CI/CD: GitHub Actions
- Registro de imagens: GitHub Container Registry (GHCR)

2. Descrição do pipeline de CI/CD

Ferramenta utilizada: GitHub Actions, com o workflow definido em '.github/workflows/ci-cd.yml'. O pipeline é disparado em push/pull_request para as branches main e develop, e também pode ser executado manualmente (workflow_dispatch).

O pipeline é dividido em 4 jobs encadeados (cada um depende do anterior):

2.1 Job 1 - build-and-test

- Sobe um serviço Oracle (gvenzl/oracle-free) como service container do próprio job, simulando o banco real (necessário pois as migrations do Flyway usam sintaxe específica do Oracle).
- Configura o JDK 21 (Temurin) com cache de dependências Maven.
- Etapa de build: 'mvn clean compile'.
- Etapa de testes automatizados: 'mvn test', apontando para o Oracle do service container.
- Publica o relatório de testes (Surefire) e o .jar gerado como artefatos do workflow.

2.2 Job 2 - docker-build-push

- Builda a imagem Docker multi-stage usando Docker Buildx, com cache via GitHub Actions.
- Publica a imagem no GitHub Container Registry (GHCR) com as tags 'sha-<commit>' e 'latest'.
- Segue o princípio 'build once, deploy many': a mesma imagem publicada aqui é reaproveitada (via docker pull) nos deploys de staging e produção.

2.3 Job 3 - deploy-staging

- Usa o Environment 'staging' do GitHub Actions.
- Gera um arquivo .env de staging e executa 'docker compose pull' + 'docker compose up -d', subindo a aplicação (perfil staging, porta 8081) junto com o banco Oracle.
- Aguarda a aplicação ficar saudável e executa um smoke test via '/actuator/health'.
- Publica os logs do ambiente como evidência e derruba a stack ao final.

2.4 Job 4 - deploy-production

- Só executa em push para a branch main e após o sucesso do deploy-staging (promoção progressiva: staging -> produção).
- Usa o Environment 'production' (pode receber 'required reviewers' para aprovação manual).
- Mesmo fluxo do staging, porém com perfil production e porta 8080.

Observação: como o projeto não depende de um servidor remoto fornecido, os ambientes de staging/produção são provisionados dentro do próprio runner do GitHub Actions via Docker Compose. Isso permite validar de forma real build da imagem, subida do banco, execução das migrations e smoke test HTTP. Em um cenário com VM/cloud disponível, os passos de 'docker compose up' seriam substituídos por um deploy remoto (SSH ou action do provedor de nuvem), reaproveitando a mesma imagem publicada no GHCR.

3. Docker: arquitetura, comandos e imagem

3.1 Arquitetura

A solução é composta por dois containers orquestrados via Docker Compose, conectados por uma rede bridge dedicada ('sustentabilidade-net'):

- app: container da API Spring Boot, construído a partir do Dockerfile multi-stage.
- db: container do Oracle Database (imagem gvenzl/oracle-free), com volume nomeado 'oracle-data' para persistência dos dados entre reinicializações.

A aplicação só inicia após o banco reportar 'healthy' (depends_on: condition: service_healthy), e todas as credenciais/configurações são injetadas via variáveis de ambiente (arquivo .env, nunca hard-coded no código ou na imagem).

3.2 Dockerfile (multi-stage)

```
# syntax=docker/dockerfile:1

# ================================================================
# Etapa 1 - Build: compila o projeto Maven e gera o .jar executavel
# ================================================================
FROM maven:3.9.9-eclipse-temurin-21 AS build
WORKDIR /app

# Copia primeiro o pom.xml para aproveitar o cache de dependencias do Docker.
# Enquanto o pom.xml nao mudar, essa camada fica em cache entre builds.
COPY pom.xml .
RUN mvn -B dependency:go-offline

# Copia o codigo-fonte e empacota a aplicacao (os testes ja rodam na etapa
# de "testes" do pipeline de CI, por isso sao pulados aqui para acelerar
# o build da imagem).
COPY src ./src
RUN mvn -B clean package -DskipTests

# ================================================================
# Etapa 2 - Runtime: imagem final, enxuta, somente com o JRE e o .jar
# ================================================================
FROM eclipse-temurin:21-jre-alpine AS runtime
WORKDIR /app

# Usuario nao-root por boas praticas de seguranca em containers
RUN addgroup -S spring && adduser -S spring -G spring

COPY --from=build /app/target/sustentabilidade-0.0.1-SNAPSHOT.jar app.jar

RUN chown -R spring:spring /app
USER spring:spring

ENV SERVER_PORT=8080
EXPOSE 8080
```

```
# Healthcheck usado pelo Docker/Compose para saber quando a aplicacao
# esta pronta (consome o endpoint do Spring Boot Actuator)
HEALTHCHECK --interval=15s --timeout=5s --start-period=40s --retries=8 \
   CMD wget -qO- http://localhost:8080/actuator/health | grep -q '"status":"UP"' || exit 1

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

3.3 Principais comandos utilizados

```
# Build local da imagem
docker build -t sustentabilidade-api:local .

# Subir a stack completa (app + banco) em background
docker compose up -d --build

# Acompanhar logs da aplicação
docker compose logs -f app

# Verificar status/healthcheck dos containers
docker compose ps
docker inspect --format '{{.State.Health.Status}}' sustentabilidade-app

# Derrubar o ambiente (containers + rede + volumes)
docker compose down -v
```

3.4 Imagem criada

A imagem final publicada no pipeline é armazenada no GitHub Container Registry com o nome 'ghcr.io/compila-ou-chora/sustentabilidade' e duas tags por execução: 'sha-<commit>' (imutável, usada efetivamente nos deploys) e 'latest' (última versão publicada). Localmente, a imagem gerada pelo docker-compose.yml recebe o nome 'sustentabilidade-api:local'.

4. Evidências de execução

4.1 Build e subida do ambiente local (app + banco Oracle)

```
NAME                  IMAGE                                COMMAND                  SERVICE   CREATED
         STATUS                        PORTS
sustentabilidade-app   sustentabilidade-api:local             "java -jar /app/app...."   app       About a
minute ago   Up About a minute (healthy)   0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp
sustentabilidade-db    gvenzl/oracle-free:23-slim-faststart   "container-entrypoin..."   db        2 minutes
ago        Up 2 minutes (healthy)        0.0.0.0:1521->1521/tcp, [::]:1521->1521/tcp
```

4.2 Smoke test da API (health check, cadastro e login)

```
$ curl http://localhost:8080/actuator/health
{"status":"UP"}

$ curl -X POST http://localhost:8080/auth/register -d '{...}'
{"id":1,"nomeEmpresa":"Eco Teste","nomeCidade":"Sao Paulo",
 "siglaEstado":"SP","email":"eco@teste.com"}

$ curl -X POST http://localhost:8080/auth/login -d '{...}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...."}
```

Os arquivos completos dessas evidências (saída de 'docker compose ps', status de health check e logs da aplicação aplicando as migrations do Flyway) estão na pasta 'docs/evidencias/' do repositório.

4.3 Pipeline no GitHub Actions (build, testes e deploy)

Exigência da entrega: comprovar que o pipeline executa, com sucesso, as quatro etapas encadeadas (build-and-test, docker-build-push, deploy-staging e deploy-production), incluindo a execução dos testes automatizados ('mvn test') no job 'build-and-test' e o smoke test HTTP contra o endpoint '/actuator/health' nos jobs de deploy.

Evidência abaixo: execução real do workflow (run #38011718907, branch 'main'), na qual os 4 jobs foram concluídos com sucesso.

Aba 'Actions' do GitHub: os 4 jobs do pipeline concluídos com sucesso (Build & Testes, Build & Push da imagem Docker, Deploy - Staging e Deploy - Producao).

Log do job 'build-and-test' - execução dos testes automatizados (mvn test)

```
$ mvn -B test
[INFO] Building sustentabilidade 0.0.1-SNAPSHOT
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
```

```
[INFO] Running br.com.fiap.sustentabilidade.SustentabilidadeApplicationTests
[INFO] Database: jdbc:oracle:thin:@localhost:1521/FREEPDB1 (Oracle 23.26)
[INFO] Migrating schema "SUSTENTABILIDADE_APP" to version "002 - create table empresa"
[INFO] ...
[INFO] Successfully applied 4 migrations to schema "SUSTENTABILIDADE_APP", now at version v5
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 7.276 s -- in
br.com.fiap.sustentabilidade.SustentabilidadeApplicationTests
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  12.652 s
```

Log do job 'deploy-staging' - smoke test em '/actuator/health' (porta 8081)

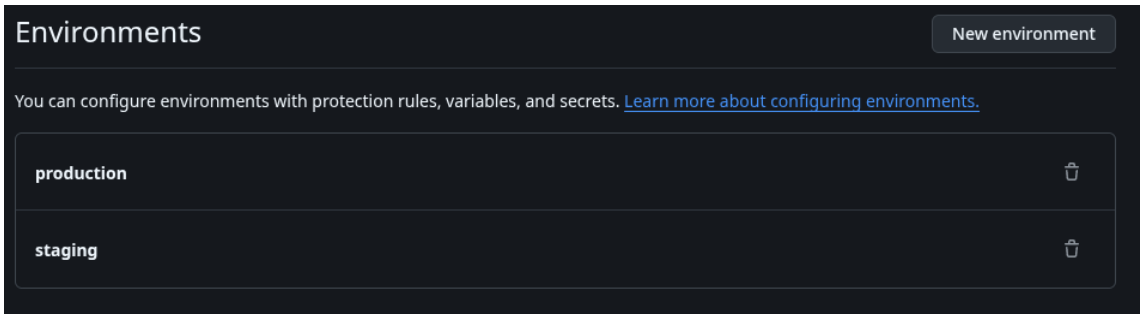
```
Aguardando aplicacao subir... (tentativa 1)
Aplicacao de staging esta UP
$ curl -f http://localhost:8081/actuator/health
{"status":"UP","components":{"db":{"status":"UP","details":
  {"database":"Oracle","validationQuery":"isValid()"}},
  "diskSpace":{"status":"UP","details":{"path":"/app/.","exists":true}},
  "ping":{"status":"UP"},"ssl":{"status":"UP","details":{"validChains":[]}}}}
100   337    0   337    0     0  26445      0 --:--:-- --:--:-- --:--:-- 28083
Smoke test de staging concluido com sucesso
```

Log do job 'deploy-production' - smoke test em '/actuator/health' (porta 8080)

```
Aguardando aplicacao subir... (tentativa 1)
Aplicacao de producao esta UP
$ curl -f http://localhost:8080/actuator/health
{"status":"UP"}
100    15    0    15    0     0   2355      0 --:--:-- --:--:-- --:--:--  2500
Smoke test de producao concluido com sucesso
```

4.4 Ambientes de staging e produção

Exigência da entrega: comprovar a existência dos environments 'staging' e 'production' configurados no repositório e o sucesso do deploy em cada um (health check retornando UP).



Settings > Environments do GitHub: ambientes 'staging' e 'production' configurados e utilizados pelos jobs de deploy.

Os logs de cada deploy (seções 4.3) mostram o health check retornando status UP tanto em staging (porta 8081) quanto em produção (porta 8080), confirmando o deploy nos dois ambientes.

5. Desafios encontrados e soluções

Configuração do pom.xml quebrada

O pom.xml original estava com 'packaging=pom' e um módulo Maven ('insomnia') que não existia no disco, o que impedia qualquer build do projeto. Foi corrigido para 'packaging=jar' e o módulo inexistente foi removido, permitindo o build normal via 'mvn package' e, consequentemente, no Dockerfile e no pipeline.

Banco de dados Oracle em containers (dev/test/CI)

A aplicação usa Oracle com sintaxe específica nas migrations do Flyway (VARCHAR2, sequences com NOCACHE/NOCYCLE), o que inviabiliza o uso de um banco genérico (H2, Postgres) nos testes. A solução foi adotar a imagem gratuita 'gvenzl/oracle-free', usada tanto no docker-compose.yml (ambiente local) quanto como service container no job de testes do GitHub Actions, garantindo paridade entre os ambientes.

Credenciais hard-coded no application.properties

O arquivo original tinha a URL/usuário do Oracle da FIAP fixos no código. Para permitir containerização e múltiplos ambientes, as propriedades de datasource, porta e segredo JWT foram parametrizadas com variáveis de ambiente (ex.: '${SPRING_DATASOURCE_URL:...}'), mantendo o valor original como fallback para não quebrar o uso fora de containers.

Exposição do health check sem autenticação

Como a aplicação usa Spring Security com JWT para quase todas as rotas, o endpoint '/actuator/health' precisou ser liberado explicitamente ('permitAll') para que o Docker Healthcheck e o smoke test do pipeline pudessem consultá-lo sem token.

Build once, deploy many

Para evitar reconstruir a imagem Docker em cada ambiente (o que poderia gerar artefatos ligeiramente diferentes entre staging e produção), o pipeline builda e publica a imagem uma única vez no GHCR, e os jobs de deploy apenas fazem 'docker pull' dessa mesma imagem.

6. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	[X]
Dockerfile funcional	[X]
docker-compose.yml ou arquivos Kubernetes	[X]
Pipeline com etapas de build, teste e deploy	[X]
README.md com instruções e prints	[X]
Documentação técnica com evidências (PDF ou PPT)	[X]
Deploy realizado nos ambientes staging e produção	[X]

O item de compactação em .zip deve ser marcado após a geração do arquivo final para envio.